```
$ chat "Hi! My name is Lukas."
```

```
$ chat "Hi! My name is Lukas."

Hi Lukas, nice to meet you – what can I
help you with?
```

VISMA

```
$ chat "What is my name?"   # fresh request
```

```
$ chat "What is my name?"   # fresh request

I don't know your name — you haven't told
me, and I don't have access to any personal
information about you.
```

VISMA

```
$ chat '[
  {"role":"user",      "content":"Hi! My name is Lukas."},
  {"role":"assistant", "content":"Hi Lukas, nice to meet
                          you – what can I help you with?"},
  {"role":"user",      "content":"What is my name?"} ]'
```

Your name is Lukas.

The spine

```
text in → text out
```

The model

What can it actually do on its own?

```
POST /v1/messages
```

The provider API

What are you actually paying for?

```
while stop_reason == "tool_use"
```

Your harness

Where does everything else live?

VISMA

```
$ chat() {
  curl -s https://api.anthropic.com/v1/messages \
    -H "x-api-key: $ANTHROPIC_API_KEY" \
    -H "anthropic-version: 2023-06-01" \
    -d "{\"model\": \"$MODEL\", \"max_tokens\": 256, \"messages\": $1}" \
    | jq -r '.content[] | select(.type=="text") | .text'
}
```

VISMA

```
$ POST https://api.anthropic.com/v1/messages

{
  "model":      "claude-opus-5",
  "max_tokens": 1024,
  "messages":   [
    {"role": "user", "content": "What is in config.py?"}
  ]
}
```

```
$ POST https://api.anthropic.com/v1/messages

{
  "model":      "claude-opus-5",
  "max_tokens": 1024,
  "tools": [{
    "name":         "read_file",
    "description":  "Read a file from disk",
    "input_schema": { "path": "string" }
  }],
  "messages":   [
    {"role": "user", "content": "What is in config.py?"}
  ]
}
```

VISMA

```
$ ask "What is in config.py?"   (tools: [read_file])
```

VISMA

```
$ ask "What is in config.py?"   (tools: [read_file])

{
  "stop_reason": "tool_use",
  "tool_use": [{
     "name":  "read_file",
     "input": { "path": "config.py" }
  }]
}
```

```
$ cat config.py           # I am the runtime

respone_timeout = 30  # planted bug: misspelled name
def get_timeout():
    return response_timeout
```

VISMA

```
$ resend the WHOLE conversation, plus the favor:

"messages": [
  {"role": "user",      "content": "What is in config.py?"},
  {"role": "assistant", "content": [ tool_use: read_file ]},
  {"role": "user",      "content": [{
     "type": "tool_result",
     "tool_use_id": "toolu_01A...",
     "content": "respone_timeout = 30 ..."}]}
]
```

VISMA

```
$ resend conversation + tool_result

stop_reason: "end_turn"
```

config.py defines a timeout setting and a get_timeout() function, but contains a bug: the variable is misspelled as respone_timeout while get_timeout() references response_timeout, so calling it raises a NameError.

VISMA

```
$ ask "Deploy to production."  (tools: [deploy_to_production])
```

VISMA

```
$ ask "Deploy to production."  (tools: [deploy_to_production])

{
  "stop_reason": "tool_use",
  "tool_use": [{ "name":  "deploy_to_production",
                 "input": { "confirm": true } }]
}
```

Nothing happened.

VISMA

`end_turn` · **`tool_use`** · `max_tokens` · `stop_sequence` · `pause_turn` · `refusal`

↑ model wants something from us

```
while stop_reason == "tool_use":
```

So we wrap it in a loop

`end_turn` · **`tool_use`** · `max_tokens` · `stop_sequence` · `pause_turn` · `refusal`

VISMA

```
while stop_reason == "tool_use":
```

`end_turn` · **`tool_use`** · `max_tokens` · `stop_sequence` · `pause_turn` · `refusal`

Now we have an agent

VISMA

```python
#!/usr/bin/env python3
"""An agent is a loop. That's it. That's the whole thing."""
import os, subprocess, sys
import anthropic

MODEL = os.environ.get("MODEL", "claude-opus-5")
TOOLS = [
    {"name": "read_file", "description": "Read a file from disk",
     "input_schema": {"type": "object", "properties": {"path": {"type": "string"}},
                      "required": ["path"]}},
    {"name": "bash", "description": "Run a shell command and return its output",
     "input_schema": {"type": "object", "properties": {"command": {"type": "string"}},
                      "required": ["command"]}},
]

def run_tool(name, args):
    if name == "read_file":
        return open(args["path"]).read()
    r = subprocess.run(args["command"], shell=True, capture_output=True, text=True, timeout=60)
    return (r.stdout + r.stderr)[-4000:] or "(no output)"

client = anthropic.Anthropic()
messages = [{"role": "user", "content": sys.argv[1]}]

for turn in range(1, 6):                                # capped: stage safety
    resp = client.messages.create(
        model=MODEL, max_tokens=4096, tools=TOOLS,
        output_config={"effort": "low"},                # snappy stage traces
        system="You are a coding agent. Use the tools to finish the task, then stop.",
        messages=messages)
    messages.append({"role": "assistant", "content": resp.content})
    print(f"--- turn {turn} (input_tokens={resp.usage.input_tokens})")
    if resp.stop_reason != "tool_use":                  # the load-bearing line
        print(next(b.text for b in resp.content if b.type == "text"))
        break
    results = []
    for block in resp.content:
        if block.type == "tool_use":
            print(f"    {block.name}({block.input})")
            results.append({"type": "tool_result", "tool_use_id": block.id,
                            "content": run_tool(block.name, block.input)})
    messages.append({"role": "user", "content": results})
```

VISMA

```python
#!/usr/bin/env python3
"""An agent is a loop. That's it. That's the whole thing."""
import os, subprocess, sys
import anthropic

MODEL = os.environ.get("MODEL", "claude-opus-5")
TOOLS = [
    {"name": "read_file", "description": "Read a file from disk",
     "input_schema": {"type": "object", "properties": {"path": {"type": "string"}},
                      "required": ["path"]}},
    {"name": "bash", "description": "Run a shell command and return its output",
     "input_schema": {"type": "object", "properties": {"command": {"type": "string"}},
                      "required": ["command"]}},
]

def run_tool(name, args):
    if name == "read_file":
        return open(args["path"]).read()
    r = subprocess.run(args["command"], shell=True, capture_output=True, text=True, timeout=60)
    return (r.stdout + r.stderr)[-4000:] or "(no output)"

client = anthropic.Anthropic()
messages = [{"role": "user", "content": sys.argv[1]}]

for turn in range(1, 6):                                # capped: stage safety
    resp = client.messages.create(
        model=MODEL, max_tokens=4096, tools=TOOLS,
        output_config={"effort": "low"},                # snappy stage traces
        system="You are a coding agent. Use the tools to finish the task, then stop.",
        messages=messages)
    messages.append({"role": "assistant", "content": resp.content})
    print(f"--- turn {turn} (input_tokens={resp.usage.input_tokens})")
    if resp.stop_reason != "tool_use":                  # the load-bearing line
        print(next(b.text for b in resp.content if b.type == "text"))
        break
    results = []
    for block in resp.content:
        if block.type == "tool_use":
            print(f"    {block.name}({block.input})")
            results.append({"type": "tool_result", "tool_use_id": block.id,
                            "content": run_tool(block.name, block.input)})
    messages.append({"role": "user", "content": results})
```

VISMA

```
$ python agent.py "make the tests pass"

--- turn 1 (input_tokens=479)
    bash({'command': 'pytest'})

FAILED test_config.py::test_timeout_is_thirty_seconds
NameError: name 'response_timeout' is not defined
1 failed in 0.01s
```

```
--- turn 2 (input_tokens=1297)
    read_file({'path': 'config.py'})
    read_file({'path': 'test_config.py'})

--- turn 3 (input_tokens=1538)
    bash({'command': "cat > config.py <<'EOF'
response_timeout = 30
def get_timeout():
    return response_timeout
EOF"})
```

```
--- turn 4 (input_tokens=1743)
    bash({'command': 'pytest'})
1 passed in 0.01s

--- turn 5 (input_tokens=1980)
Tests pass now. ✅  config.py defined the variable as
respone_timeout but get_timeout() returned
response_timeout, causing a NameError. Renamed it.
```

$ what actually goes over the wire

that gap is the product

	agent.py	Claude Code
tools	2	28
system	68 chars	27,432 chars
loop	42 lines	the same loop

Same endpoint. Same protocol. Same loop.

VISMA

The spine

```
text in → text out
```

The model

Weights. Tokens in, tokens out. No memory, no side effects.

```
POST /v1/messages
```

The provider API

One endpoint. The tool-call protocol. Streaming, caching, billing.

```
while stop_reason == "tool_use"
```

Your harness

The loop. The actual tools. System prompt, permissions, context management, UI, subagents, MCP. Just software.

VISMA

Last 5% is the product

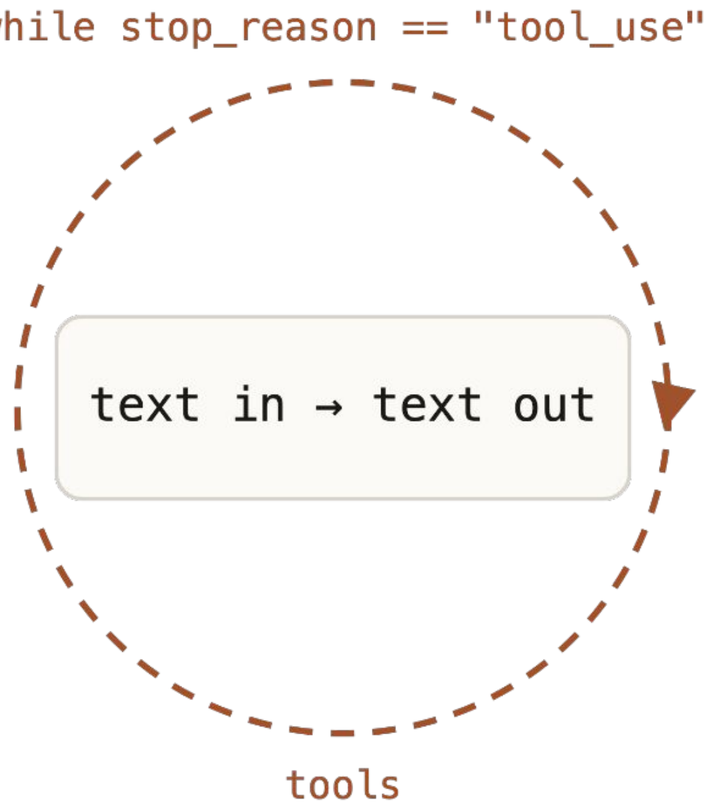
permissions	what may run without asking
context mgmt	what to forget, and when
error recovery	the model **will** emit garbage
subagents	a tool whose handler starts another loop
mcp	someone else's tools, same JSON

VISMA

Claude Code

your future internal tool

Cursor



OpenAI Agents SDK

Copilot agent mode

LangChain / LangGraph

VISMA

You have API credentials.

Now what?



github.com/lukasvavrek/api-credentials-now-what

the 42 lines · the capture scripts · the spy proxy · these slides

VISMA